Another option is to modify the design of your data structure and its invariants so that modifications are done as a series of indivisible changes, each of which preserves the invariants. This is generally referred to as *lock-free programming* and is difficult to get right. If you're working at this level, the nuances of the memory model and identifying which threads can potentially see which set of values can get complicated. The memory model is covered in chapter 5, and lock-free programming is discussed in chapter 7.

Another way of dealing with race conditions is to handle the updates to the data structure as a *transaction*, just as updates to a database are done within a transaction. The required series of data modifications and reads is stored in a transaction log and then committed in a single step. If the commit can't proceed because the data structure has been modified by another thread, the transaction is restarted. This is termed *software transactional memory (STM)*, and it's an active research area at the time of writing. This won't be covered in this book, because there's no direct support for STM in C++. However, the basic idea of doing something privately and then committing in a single step is something that I'll come back to later.

The most basic mechanism for protecting shared data provided by the C++ Standard is the *mutex*, so we'll look at that first.

3.2. Protecting shared data with mutexes

So, you have a shared data structure such as the linked list from the previous section, and you want to protect it from race conditions and the potential broken invariants that can ensue. Wouldn't it be nice if you could mark all the pieces of code that access the data structure as *mutually exclusive*, so that if any thread was running one of them, any other thread that tried to access that data structure had to wait until the first thread was finished? That would make it impossible for a thread to see a broken invariant except when it was the thread doing the modification.

Well, this isn't a fairy tale wish—it's precisely what you get if you use a synchronization primitive called a *mutex* (*mut*ual *ex*clusion). Before